

Logica Sequenziale

Logica Combinatoria – Logica Sequenziale

I circuiti costruiti nelle precedenti unità sono chiamati **combinatori**. Questo significa **che l'output è solamente una combinazione** che dipende dagli ingressi.

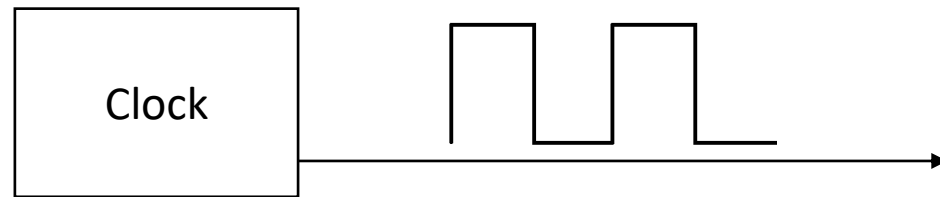
Questi circuiti possono arrivare a svolgere operazioni di una certa complessità (come la ALU) tuttavia non hanno la capacità di mantenere (memorizzare) uno stato o un valore.

Dato che i computer non devono solamente svolgere calcoli ma anche salvare e recuperare valori, i circuiti che abbiamo sviluppato nell'unità didattica precedente devono essere completati **con circuiti che tengono in memoria i dati nel tempo**. Tali circuiti vengono chiamati **Sequenziali**.

Clock

Nei computer il passaggio del tempo è rappresentato da un clock che fornisce un'onda quadra periodica.

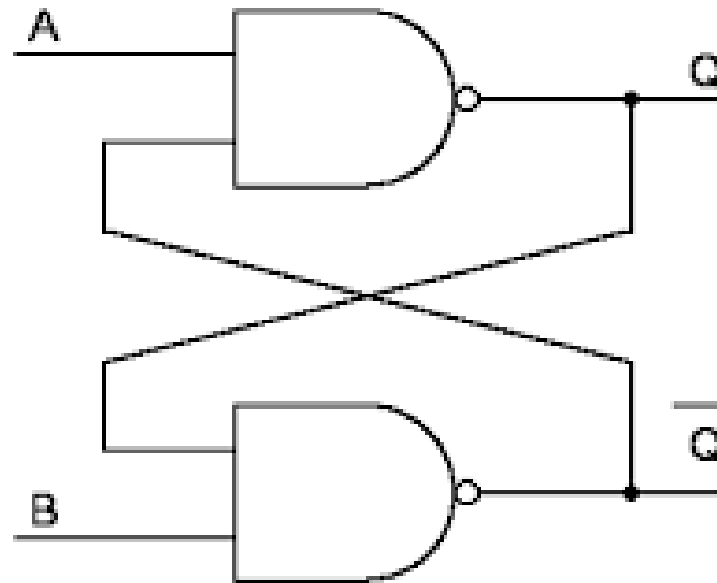
- Tale circuito è formato da un oscillatore che genera l'onda quadra. Per semplicità verrà disegnato con un blocco.



- Il segnale generato dal clock viene mandato a tutti i circuiti della CPU che ne hanno bisogno

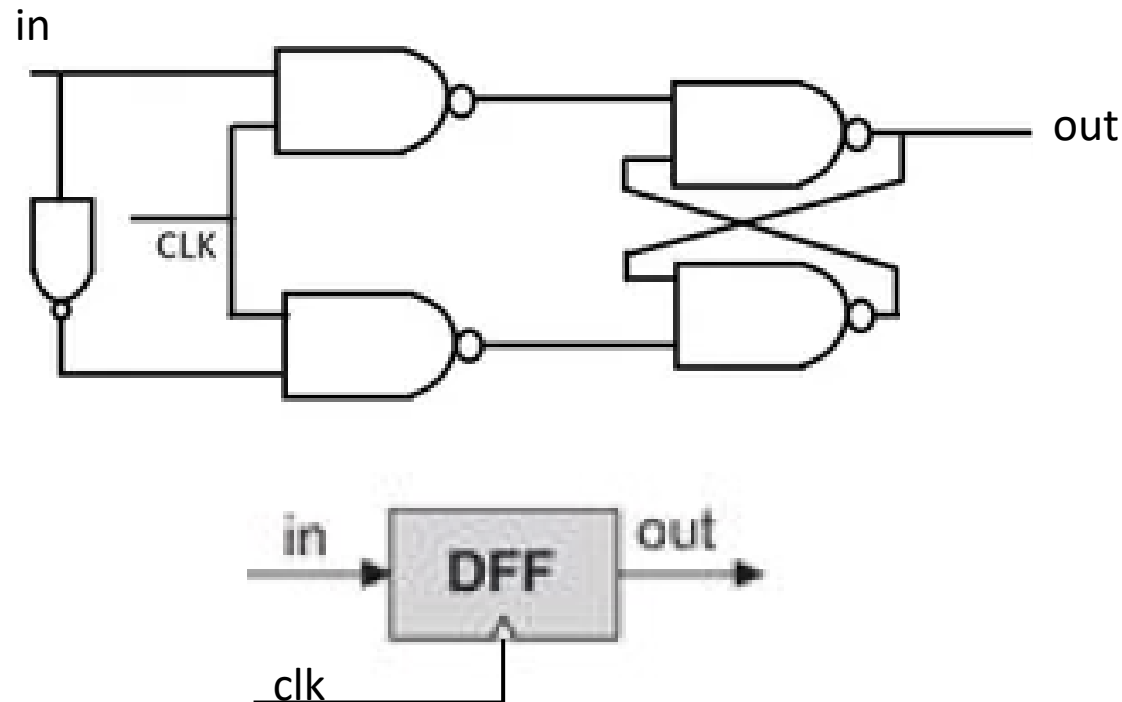
Flip-Flop

E' possibile collegare le porte logiche utilizzate fino ad ora per ottenere un componente con memoria. Ecco un esempio:



Flip-Flop tipo D

Applicando un ulteriore miglioramento alla rete precedente si ottiene la seguente porta logica:



Flip-Flop tipo D

Nei computer il componente con memoria elementare viene chiamato flip-flop e ne esistono molte varianti. Nella nostra unità didattica utilizzeremo la variante Data, chiamata anche Data Flip Flop o DFF la cui interfaccia consiste in un singolo bit di ingresso e un singolo bit di uscita e il segnale di clock.



Tale dispositivo mette come output il valore di ingresso dell'istante precedente.

$$out(t) = in(t - 1)$$

Tale dispositivo è alla base dei registri e delle memorie contenuti in un computer.

Registri

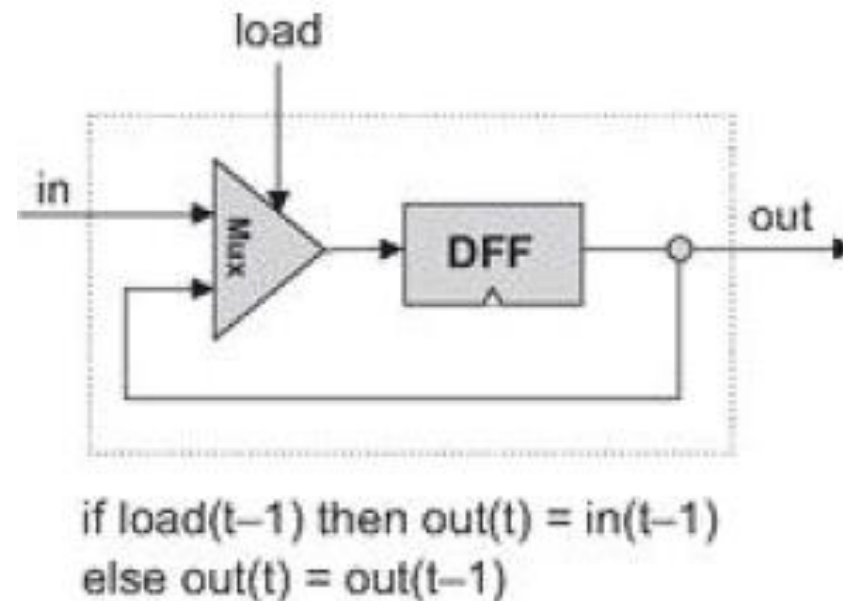
Un registro è un dispositivo di stoccaggio dati che esegue due operazioni elementari:

- Conserva un dato
- Carica un nuovo dato al suo interno (e perde il dato vecchio)

Il registro più semplice possibile memorizza un bit.

Registri

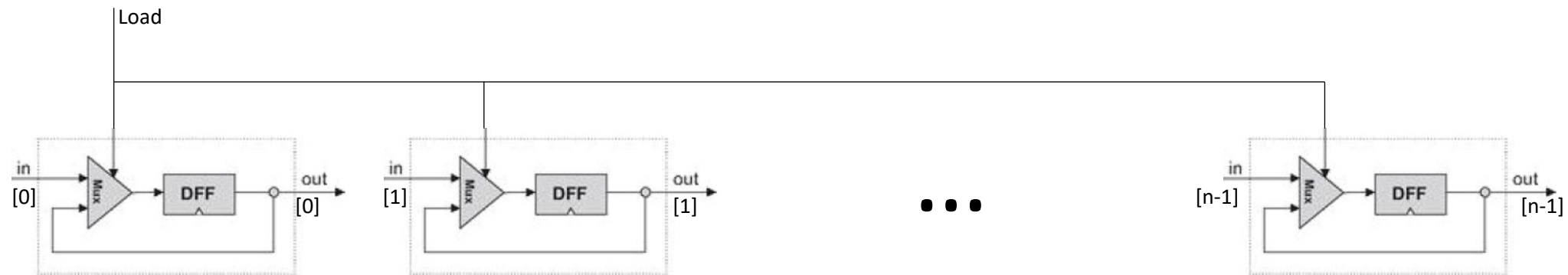
Un registro è un dispositivo di stoccaggio dati che può ricordare un dato o salvarne uno nuovo al suo interno. Si ottiene combinando un multiplexer e un flip flop di tipo data.



One bit register

Registri

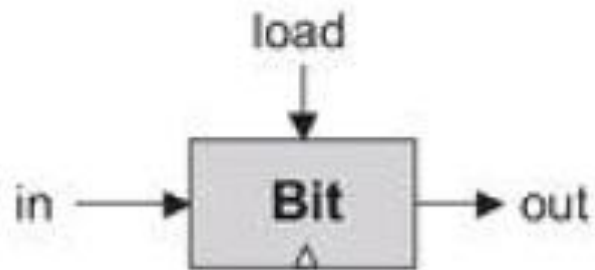
A questo punto è possibile creare registri di grandezza a nostro piacimento mettendo insieme n registri da un bit.



N-bit register

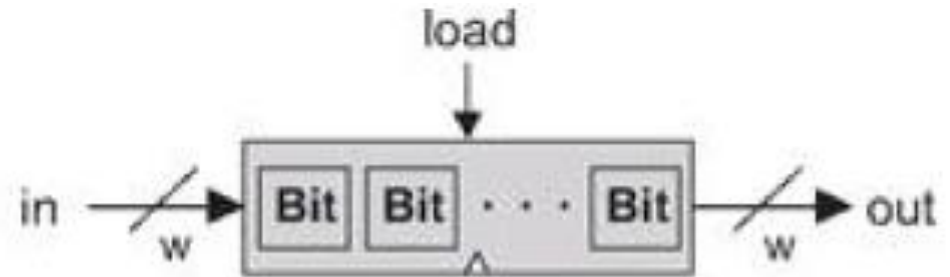
Registri

Per semplicità i circuiti mostrati nelle slide successive verranno semplificati nel seguente modo:



if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

Binary cell (Bit)



if load($t-1$) then out(t) = in($t-1$)
else out(t) = out($t-1$)

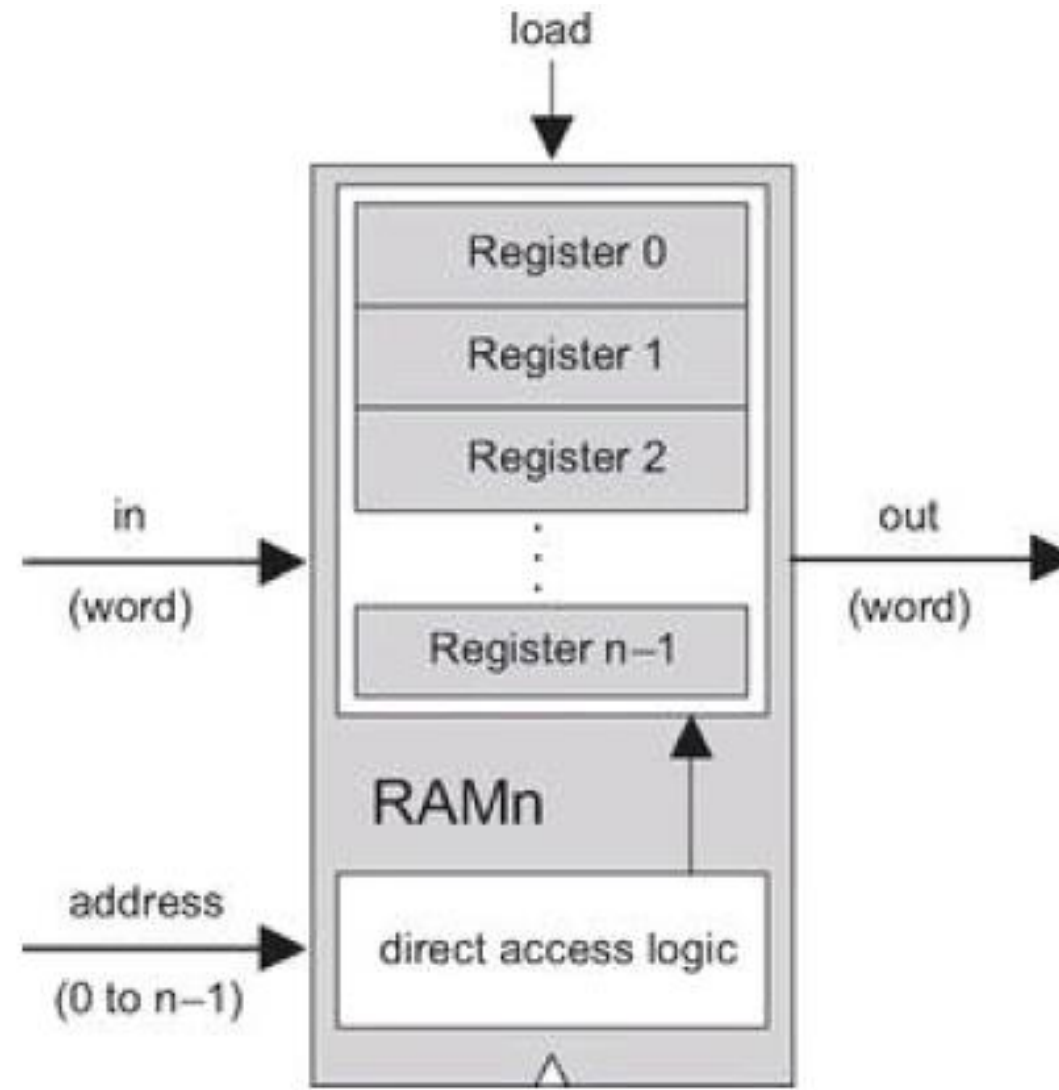
w-bit register

Memorie

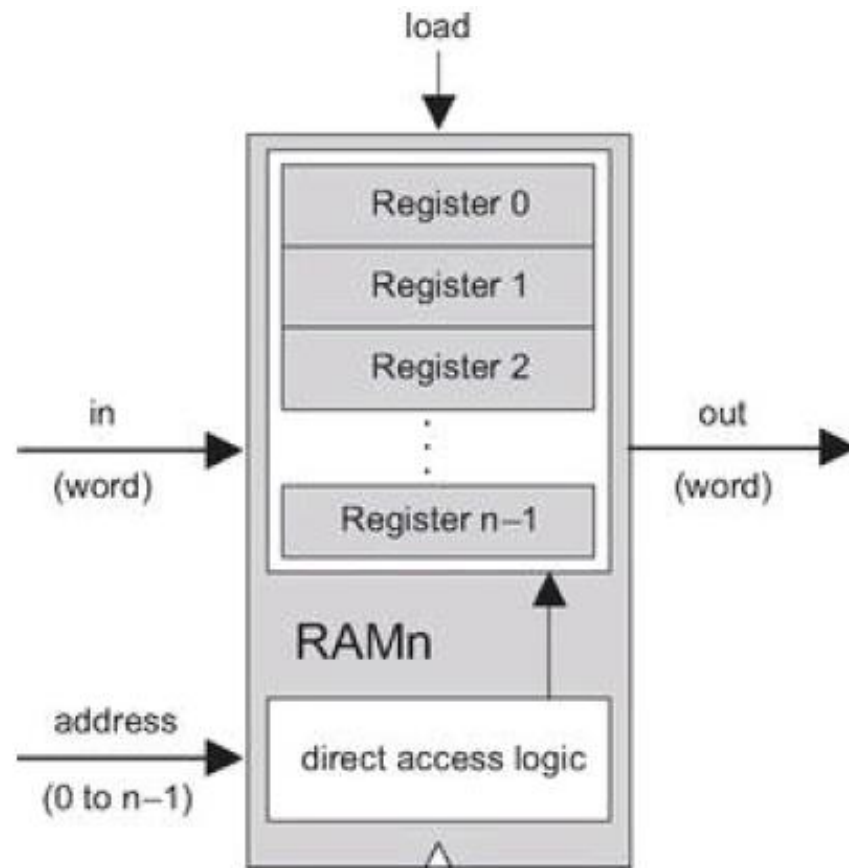
È possibile creare memorie partendo dai registri nel seguente modo:

- Si definisce la lunghezza del registro (che viene anche chiamato word)
- Si assegna ad ogni registro un indirizzo
- Si crea una rete logica in grado di attivare solamente quel registro una volta che viene dato in ingresso un indirizzo.

Memorie



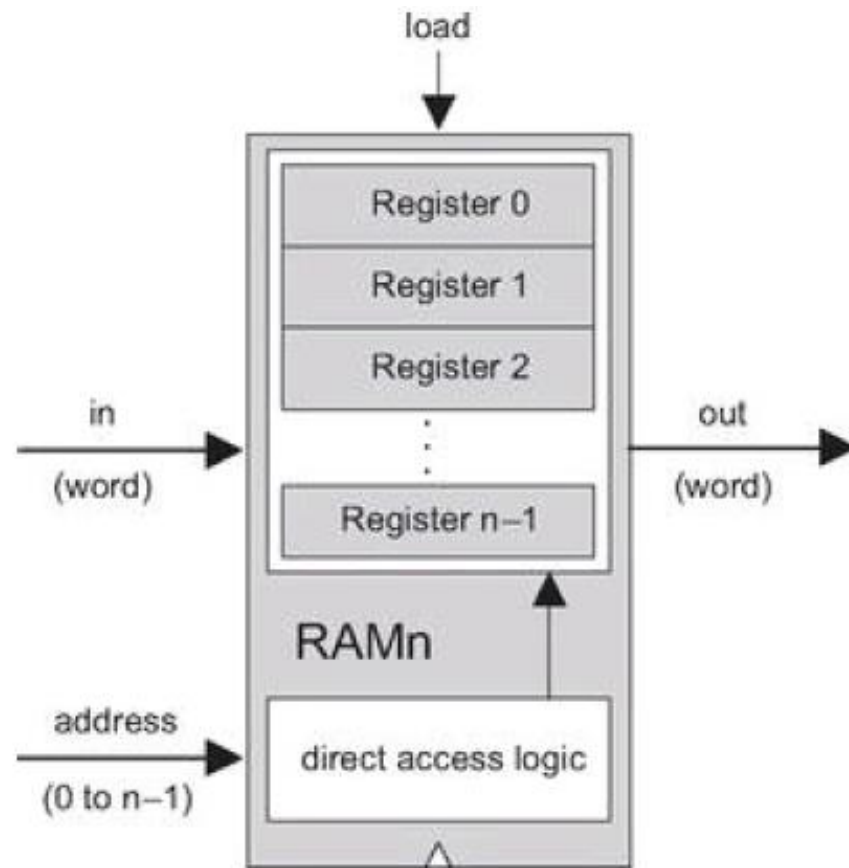
Memorie



Utilizzo della memoria in lettura:

- Metto load=0
- Fornisco un indirizzo di ingresso
- Al prossimo segnale di clock la memoria manda il dato in out
- Il segnale in non viene utilizzato

Memorie



Utilizzo della memoria in scrittura:

- Metto un dato su in
- Metto load=1
- Fornisco un indirizzo di ingresso
- Al prossimo segnale di clock la memoria salva il dato all'indirizzo fornito in ingresso.
- Il segnale out non viene utilizzato

Contatori

Il contatore è un registro la cui quantità viene incrementata di 1 ad ogni ciclo di clock.

Uno dei contatori più utilizzati è il Program Counter il cui contenuto viene interpretato come: "indirizzo della prossima istruzione da eseguire".

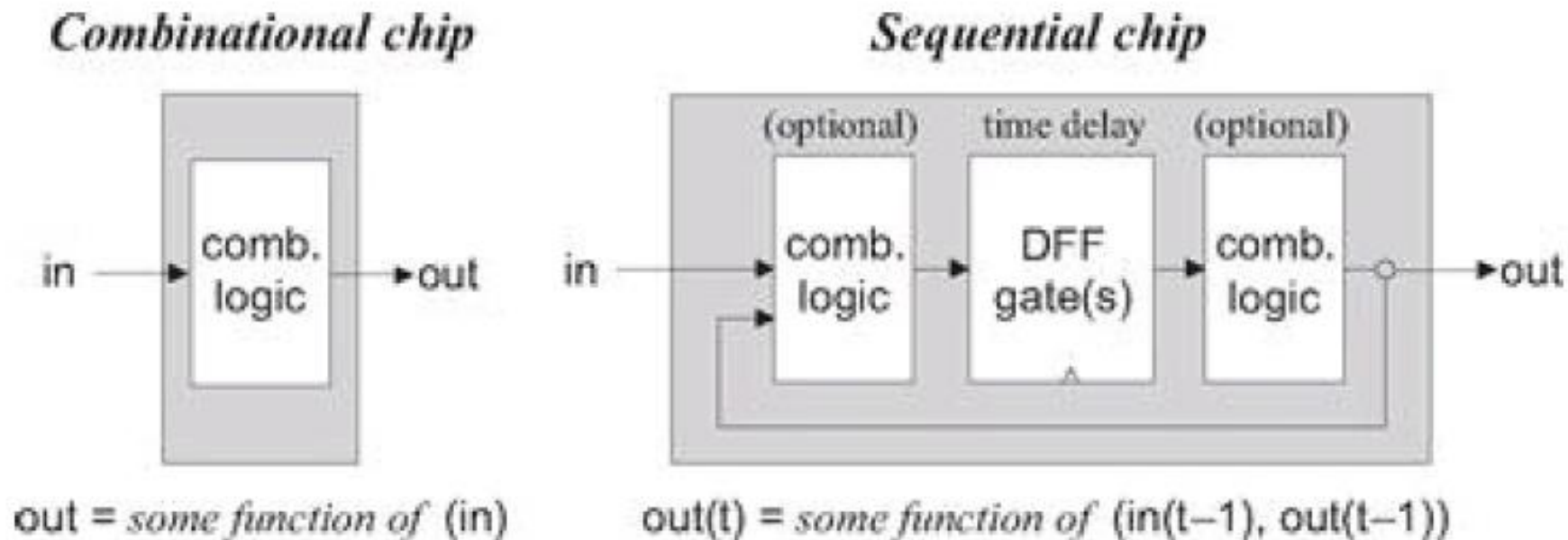
Un registro di tipo contatore si crea combinando la logica di un registro normale con un circuito sommatore.

Viene equipaggiato inoltre con altre funzionalità come il reset a zero o la possibilità di caricare un indirizzo ben preciso.

Differenza tra sequenziale e combinatorio

Combinatorio: l'uscita dipende solo dall'ingresso

Sequenziale: l'uscita dipende sia dall'ingresso che dallo stato attuale, entra in gioco il fattore tempo



Data Flip Flop

L'unità elementare di qualsiasi dispositivo sequenziale è il flip flop, lo tratteremo esattamente come la porta NAND quando abbiamo costruito le porte logiche più avanzate.



Chip name: DFF

Inputs: in

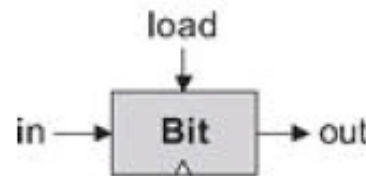
Outputs: out

Function: $out(t) = in(t-1)$

Comment: This clocked gate has a built-in implementation and thus there is no need to implement it.

1-Bit Register

Registro ad un bit che consente di salvare e leggere un valore binario.



Chip name: Bit

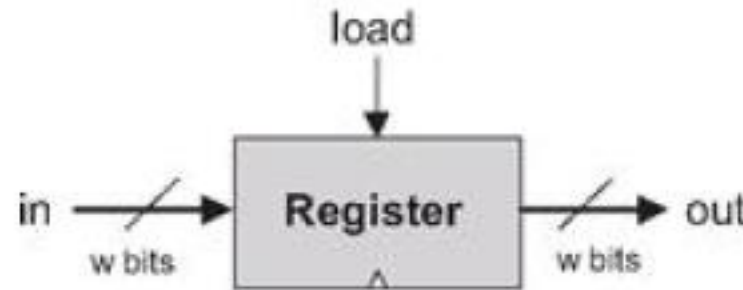
Inputs: in, load

Outputs: out

Function: If $\text{load}(t-1)$ then $\text{out}(t) = \text{in}(t-1)$
else $\text{out}(t) = \text{out}(t-1)$

Register

Registro a 16 bit composto da 8 1-Bit Register.



Chip name: Register

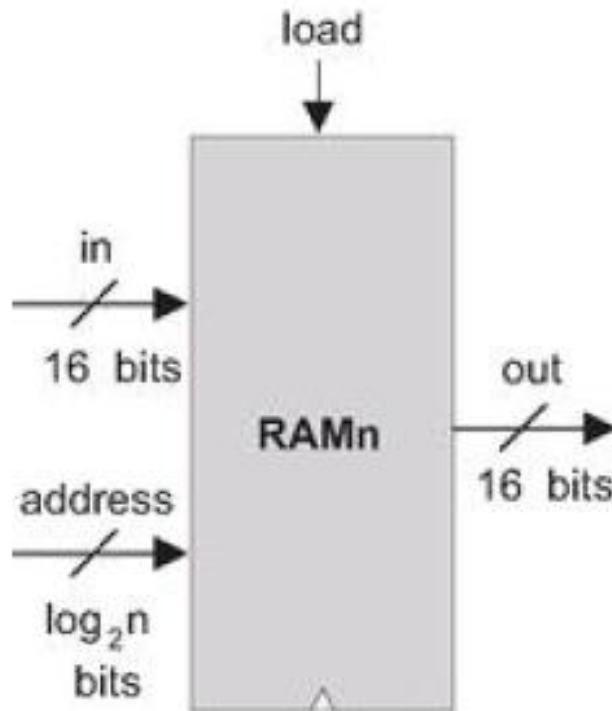
Inputs: in[16], load

Outputs: out[16]

Function: If load(t-1) then out(t)=in(t-1)
else out(t)=out(t-1)

Memory

Insieme di Registri con una rete logica in grado di attivare solamente il registro desiderato a seconda dell'indirizzo in ingresso.



Chip name: RAMn // n and k are listed below

Inputs: in[16], address[k], load

Outputs: out[16]

Function: $out(t) = RAM(address(t))(t)$

If load(t-1) then

$RAM[address(t-1)](t) = in(t-1)$

Comment: "=" is a 16-bit operation.

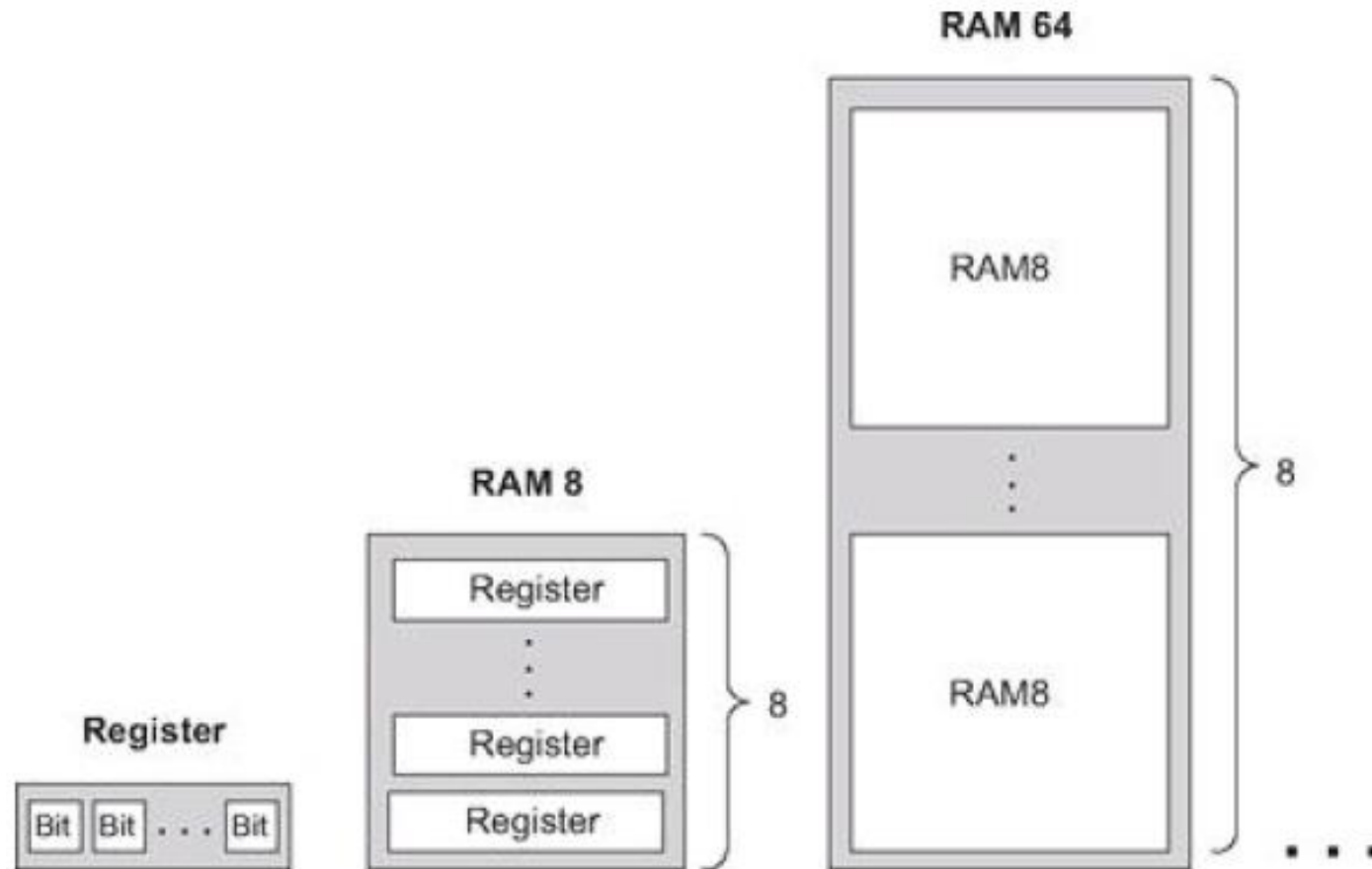
The specific RANI chips needed for the Hack platform are:

Chip name

RAM8	8	3
RAM64	64	6
RAM512	512	9
RAM4K	4096	12
RAM16K	16384	14

Memory

E' possibile costruire una memoria procedendo per piccoli step:



Con altri tre passaggi simili si arriva ad ottenere una memoria RAM da 16K

Program Counter

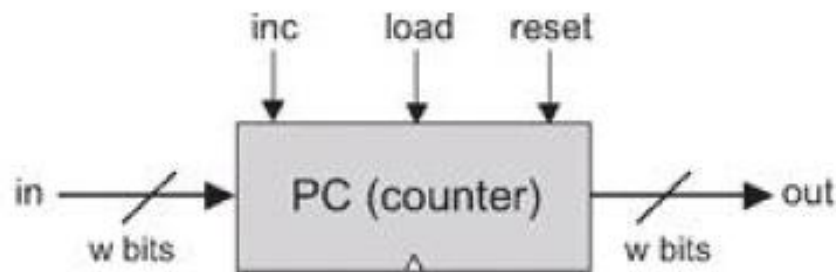
Registro utilizzato per contenere un numero ed incrementarlo in caso di necessità. Il suo contenuto può anche essere azzerato o fissato ad un valore ben preciso.

Chip name: RAMn // n and k are listed below

Inputs: in[16), address[k], load

Outputs: out[16]

Function: If reset(t-1) then out(t)=0
else if load(t-1) then out(t)=in(t-1)
else if inc(t-1) then out(t)=out(t-1)+1
else out(t)=out(t-1)



Program Counter

Esempio di funzionamento del PC (Program Counter)

